What's Next?

As I mentioned at the end of Chapter 5, *Render Native Screens with SwiftUI*, you can use this approach to build any native screen or interaction you want. You could swap out the map and render data from the Health Platform API. Or convert the list of hikes to native elements and cache it for offline access. The list goes on.

But because of how expensive native screens are to build, each native feature should only be reserved for the most critical pieces of your app.

The good news is that there is a middle ground. When you want a little extra fidelity, you can upgrade individual *components* to native without committing to an entire SwiftUI or Jetpack Compose screen. The next chapter covers how to integrate bridge components to add native buttons to your app.

A heads up that even if you're only working through the Android side of things, you'll still need to read through Chapter 7, *Build iOS Bridge Components with Swift*. It introduces concepts and code that are required for building bridge components with Android.

Footnotes

[31] https://developer.android.com/compose

[32] https://developer.android.com/jetpack/androidx/releases/compose-kotlin

[33] https://developers.google.com/maps/documentation/android-sdk/get-api-key

[34] https://youtu.be/2_HZObVbe-g

[35] https://developers.google.com/maps/documentation/android-sdk/secrets-gradle-plugin

[36] https://developers.google.com/maps/documentation/android-sdk/maps-compose